

20221528 홍성주

자료구조 Data Structure

정렬 알고리즘 구현 과제

(Insertion Sorting, Selection Sorting,
Merge Sorting, Quick Sorting)

[소스코드]

```
import java.util.Random;
import java.util.Scanner;

public class sort {

    // insertionSort: 앞쪽부터 정렬된 영역에 현재 값을 적절한 위치에 삽입
    public static void insertionSort(int[] arr){
        int index;
        int temp;
        for(int i = 1 ; i < arr.length ; i++){
            index = i;
            for(int j = i - 1 ; j >= 0 ; j--){
                // 앞의 값들 중에서 현재 값보다 큰 값 찾기
                if(arr[i] < arr[j]) index = j;
                else break;
            }
            temp = arr[i];
            if(index != i){
                // index부터 오른쪽으로 한 칸씩 밀고, arr[i] 삽입
                System.arraycopy(arr,index,arr,index+1,i-index);
                arr[index] = temp;
            }
            // 정렬 단계 출력
            for(int k = 0 ; k < arr.length ; k++) System.out.print(arr[k]+" ");
            System.out.println();
        }
    }
    print(arr); // 최종 결과 출력
}

// selectionSort: 가장 큰 값을 찾아 뒤쪽부터 채워나감
public static void selectionSort(int[] arr){
    int big;
    int temp;
    for(int i = arr.length-1 ; i > 0 ; i--){
        big = 0;
        for(int j = 1 ; j <= i ; j++){
            if(arr[j] > arr[big]){
                big = j;
            }
        }
        // 현재 i 위치에 가장 큰 값을 넣음
        temp = arr[i];
        arr[i] = arr[big];
        arr[big] = temp;
        // 정렬 단계 출력
        for(int k = 0 ; k < arr.length ; k++) System.out.print(arr[k]+" ");
        System.out.println();
    }
    print(arr); // 최종 결과 출력
}

// mergeSort: 외부에서 호출되는 정렬 함수
public static void mergeSort(int[] arr) {
    mergeSort2(arr, 0, arr.length); // 내부 재귀 함수 호출
    print(arr); // 최종 결과 출력
}

// mergeSort 재귀 함수
private static void mergeSort2(int[] arr, int first, int n) {
    int n1, n2;
    if (n > 1) {
        n1 = n / 2;
```

```

        n2 = n - n1;
        // 배열을 반으로 나누어 각각 정렬
        mergeSort2(arr, first, n1);
        mergeSort2(arr, first + n1, n2);
        // 정렬된 두 배열 병합
        merge(arr, first, n1, n2);
        // 병합 단계 출력
        for (int i = first; i < first + n; i++) System.out.print(arr[i] + " ");
        System.out.println();
    }
}

// merge 함수: 두 정렬된 구간을 하나로 합침
private static void merge(int[] arr, int first, int n1, int n2) {
    int[] temp = new int[n1 + n2];
    int cp = 0, cp1 = 0, cp2 = 0;

    while (cp1 < n1 && cp2 < n2) {
        if (arr[first + cp1] <= arr[first + n1 + cp2]) temp[cp++] = arr[first + cp1++];
        else temp[cp++] = arr[first + n1 + cp2++];
    }

    while (cp1 < n1) temp[cp++] = arr[first + cp1++];
    while (cp2 < n2) temp[cp++] = arr[first + n1 + cp2++];

    // 병합 결과를 원래 배열로 복사
    for (int i = 0; i < n1 + n2; i++) arr[first + i] = temp[i];
}

// quicksort: 외부에서 호출되는 정렬 함수
public static void quicksort(int[] arr) {
    quicksort(arr, 0, arr.length); // 내부 재귀 함수 호출
    print(arr); // 최종 결과 출력
}

// quicksort 재귀 함수
private static void quicksort(int[] arr, int first, int n) {
    int pivotIndex, n1, n2;
    if (n > 1) {
        pivotIndex = arrayDivid(arr, first, n); // 분할 기준 인덱스
        n1 = pivotIndex - first;
        n2 = n - n1 - 1;
        quicksort(arr, first, n1); // 왼쪽 정렬
        quicksort(arr, pivotIndex + 1, n2); // 오른쪽 정렬

        // 분할 후 단계 출력
        for (int i = first; i < first + n; i++) System.out.print(arr[i] + " ");
        System.out.println();
    }
}

// arrayDivid 함수 (배열을 pivot 기준으로 둘로 나눔)
private static int arrayDivid(int[] arr, int first, int n) {
    int pivot = arr[first]; // pivot: 배열의 첫 번째 값
    int tooBigIndex = first + 1;
    int tooSmallIndex = first + n - 1;

    // 두 포인터가 만나기 전까지 반복
    while (tooBigIndex <= tooSmallIndex) {
        while (tooBigIndex <= tooSmallIndex && arr[tooBigIndex] <= pivot) tooBigIndex++;
        while (arr[tooSmallIndex] > pivot) tooSmallIndex--;
        if (tooBigIndex < tooSmallIndex) {
            // 큰 값과 작은 값 교환
            int temp = arr[tooBigIndex];
            arr[tooBigIndex] = arr[tooSmallIndex];

```

```

        arr[tooSmallIndex] = temp;
    }
}

// pivot을 올바른 위치로 이동
arr[first] = arr[tooSmallIndex];
arr[tooSmallIndex] = pivot;
return tooSmallIndex; // pivot 위치 반환
}

// 정렬 완료 후 출력 함수
public static void print(int[] arr){
    System.out.print("정렬 완료 -> [");
    for(int i = 0 ; i < arr.length ; i++) System.out.print(arr[i]+" ");
    System.out.println("]");
}

// 메인 함수: 정렬 테스트 프로그램
public static void main(String args[]){
    Scanner sc = new Scanner(System.in);
    Random ran = new Random();

    int[] arr = new int[32];    // 원본 배열
    int[] cArr = new int[32];   // 정렬용 복사 배열

    // 배열에 0~31 사이의 정수 32개 랜덤 저장
    for(int i = 0 ; i < arr.length ; i++) arr[i] = ran.nextInt(32);

    System.out.println("<랜덤 배열 정렬 프로그램>");
    int n = 0;
    while(n < 5){
        // 배열 출력
        System.out.print("생성된 랜덤 배열 : [ ");
        for(int i = 0 ; i < arr.length ; i++) System.out.print(arr[i]+" ");
        System.out.println("]");
        // 메뉴 출력
        System.out.print("1.Insertion Sort 2.Selection Sort 3.Merge Sort 4.Quick Sort 5.quit 입력 : ");
        n = sc.nextInt();
        // 원본 배열을 복사하여 정렬
        System.arraycopy(arr,0,cArr,0,arr.length);
        switch (n){
            case 1:
                insertionSort(cArr);
                break;
            case 2:
                selectionSort(cArr);
                break;
            case 3:
                mergeSort(cArr);
                break;
            case 4:
                quicksort(cArr);
                break;
        }
    }
    System.out.print("\n배열 정렬 프로그램을 종료합니다.");
    sc.close();
}
}

```

[출력화면]

1. Insertion Sorting 진행

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.5\lib\
<랜덤 배열 정렬 프로그램>
```

```
생성된 랜덤 배열 : [ 5 26 27 18 29 6 11 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30 ]
```

```
1.Insertion Sort 2.Selection Sort 3.Merge Sort 4.Quick Sort 5.quit 입력 : 1
```

```
5 18 26 27 29 6 11 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 18 26 27 29 11 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 11 18 26 27 29 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 11 18 18 26 27 29 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 11 12 18 18 26 27 29 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 11 12 18 18 26 27 27 29 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 11 12 18 18 22 26 27 27 29 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
5 6 11 12 18 18 22 23 26 27 27 29 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 11 12 18 18 22 23 26 27 27 29 30 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 10 11 12 18 18 22 23 26 27 27 29 30 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 8 10 11 12 18 18 22 23 26 27 27 29 30 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 8 8 10 11 12 18 18 22 23 26 27 27 29 30 31 16 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 8 8 10 11 12 16 18 18 22 23 26 27 27 29 30 31 12 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 8 8 10 11 12 12 16 18 18 22 23 26 27 27 29 30 31 28 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 8 8 10 11 12 12 16 18 18 22 23 26 27 27 28 29 30 31 6 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 6 8 8 10 11 12 12 16 18 18 22 23 26 27 27 28 29 30 31 21 26 31 2 10 5 27 8 16 30
```

```
2 5 6 6 8 8 10 11 12 12 16 18 18 21 22 23 26 27 27 28 29 30 31 26 31 2 10 5 27 8 16 30
```

```
2 5 6 6 8 8 10 11 12 12 16 18 18 21 22 23 26 26 27 27 28 29 30 31 31 2 10 5 27 8 16 30
```

```
2 2 5 6 6 8 8 10 11 12 12 16 18 18 21 22 23 26 26 27 27 28 29 30 31 31 10 5 27 8 16 30
```

```
2 2 5 6 6 8 8 10 10 11 12 12 16 18 18 21 22 23 26 26 27 27 28 29 30 31 31 5 27 8 16 30
```

```
2 2 5 5 6 6 8 8 10 10 11 12 12 16 18 18 21 22 23 26 26 27 27 28 29 30 31 31 27 8 16 30
```

```
2 2 5 5 6 6 8 8 10 10 11 12 12 16 18 18 21 22 23 26 26 27 27 27 28 29 30 31 31 8 16 30
```

```
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 18 18 21 22 23 26 26 27 27 27 28 29 30 31 31 16 30
```

```
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 31 31 30
```

```
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
```

```
정렬 완료 -> [ 2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31 ]
```

2. Selection Sorting 진행

```
실행 sort ×
↺
↻
📷
📄
🗑️
:
↑
↓
↕
↔
🖨️
🗑️
생성된 랜덤 배열 : [ 5 26 27 18 29 6 11 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30 ]
1.Insertion Sort 2.Selection Sort 3.Merge Sort 4.Quick Sort 5.quit 입력 : 2
5 26 27 18 29 6 11 18 12 27 22 23 30 2 10 8 30 8 16 12 28 6 21 26 31 2 10 5 27 8 16 31
5 26 27 18 29 6 11 18 12 27 22 23 30 2 10 8 30 8 16 12 28 6 21 26 16 2 10 5 27 8 31 31
5 26 27 18 29 6 11 18 12 27 22 23 8 2 10 8 30 8 16 12 28 6 21 26 16 2 10 5 27 30 31 31
5 26 27 18 29 6 11 18 12 27 22 23 8 2 10 8 27 8 16 12 28 6 21 26 16 2 10 5 30 30 31 31
5 26 27 18 5 6 11 18 12 27 22 23 8 2 10 8 27 8 16 12 28 6 21 26 16 2 10 29 30 30 31 31
5 26 27 18 5 6 11 18 12 27 22 23 8 2 10 8 27 8 16 12 10 6 21 26 16 2 28 29 30 30 31 31
5 26 2 18 5 6 11 18 12 27 22 23 8 2 10 8 27 8 16 12 10 6 21 26 16 27 28 29 30 30 31 31
5 26 2 18 5 6 11 18 12 16 22 23 8 2 10 8 27 8 16 12 10 6 21 26 27 27 28 29 30 30 31 31
5 26 2 18 5 6 11 18 12 16 22 23 8 2 10 8 26 8 16 12 10 6 21 27 27 27 28 29 30 30 31 31
5 21 2 18 5 6 11 18 12 16 22 23 8 2 10 8 26 8 16 12 10 6 26 27 27 27 28 29 30 30 31 31
5 21 2 18 5 6 11 18 12 16 22 23 8 2 10 8 6 8 16 12 10 26 26 27 27 27 28 29 30 30 31 31
5 21 2 18 5 6 11 18 12 16 22 10 8 2 10 8 6 8 16 12 23 26 26 27 27 27 28 29 30 30 31 31
5 21 2 18 5 6 11 18 12 16 12 10 8 2 10 8 6 8 16 22 23 26 26 27 27 27 28 29 30 30 31 31
5 16 2 18 5 6 11 18 12 16 12 10 8 2 10 8 6 8 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 16 2 8 5 6 11 18 12 16 12 10 8 2 10 8 6 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 16 2 8 5 6 11 6 12 16 12 10 8 2 10 8 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 11 6 12 16 12 10 8 2 10 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 11 6 12 10 12 10 8 2 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 11 6 2 10 12 10 8 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 11 6 2 10 8 10 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 10 6 2 10 8 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 8 6 2 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 8 2 8 5 6 8 6 2 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 2 2 8 5 6 8 6 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 2 2 6 5 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 2 2 6 5 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 2 2 6 5 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
5 2 2 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
정렬 완료 -> [ 2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31 ]
```

3. Merge Sorting 진행

```
실행 sort ×
[ 5 26 27 18 29 6 11 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30 ]
1.Insertion Sort 2.Selection Sort 3.Merge Sort 4.Quick Sort 5.quit 입력 : 3
5 26
18 27
5 18 26 27
6 29
11 18
6 11 18 29
5 6 11 18 18 26 27 29
12 27
22 23
12 22 23 27
2 30
8 10
2 8 10 30
2 8 10 12 22 23 27 30
2 5 6 8 10 11 12 18 18 22 23 26 27 27 29 30
8 31
12 16
8 12 16 31
6 28
21 26
6 21 26 28
6 8 12 16 21 26 28 31
2 31
5 10
2 5 10 31
8 27
16 30
8 16 27 30
2 5 8 10 16 27 30 31
2 5 6 8 8 10 12 16 16 21 26 27 28 30 31 31
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
정렬 완료 -> [ 2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31 ]
```

4. Quick Sorting 진행

```
실행  sort x
[ 5 26 27 18 29 6 11 18 12 27 22 23 30 2 10 8 31 8 16 12 28 6 21 26 31 2 10 5 27 8 16 30 ]
1.Insertion Sort 2.Selection Sort 3.Merge Sort 4.Quick Sort 5.quit 입력 : 4
2 2 5
8 8
6 6 8 8
11 12
11 12 12 16
10 10 11 12 12 16
6 6 8 8 8 10 10 11 12 12 16
18 18
6 6 8 8 8 10 10 11 12 12 16 16 18 18
22 23 26
6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26
27 27
27 27 27
27 27 27 28
6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28
30 30 31
30 30 31 31
6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31
정렬 완료 -> [2 2 5 5 6 6 8 8 8 10 10 11 12 12 16 16 18 18 21 22 23 26 26 27 27 27 28 29 30 30 31 31 ]
```